

**«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

**ОТЧЁТ
ПО УЧЕБНОЙ ПРАКТИКЕ**

Тема: Генетические алгоритмы. Решение sudoku

Студент гр. 4341

К.Э. Баженов

Руководитель

Т.Р. Жангиров

Санкт-Петербург

2026

ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ

Студент: К.Э. Баженов

Группа: 4341

Тема практики: Генетические алгоритмы. Решение sudoku

Задание на практику:

Решение sudoku

Для заданного игрового поля sudoku, где уже заданы цифры (размеры 9x9), необходимо разместить цифры от 1 до 9, так, чтобы они не нарушали правил.

Сроки прохождения практики: 06.07.2026 – 18.07.2026

Дата сдачи отчета: 17.07.2026

Дата защиты отчета: 17.07.2026

Студент

К.Э. Баженов

Руководитель

Т.Р. Жангиров

АННОТАЦИЯ

Работа посвящена разработке программы для решения sudoku с помощью генетического алгоритма. Рассмотрены способы представления, оценки и поиска решений, а также создан графический интерфейс для настройки параметров и наблюдения за работой алгоритма.

SUMMARY

This work focuses on the development of a program for solving Sudoku using a genetic algorithm. It examines methods for representing, evaluating, and searching for solutions, and presents a graphical user interface for configuring parameters and monitoring the algorithm's operation.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1 ПОСТАНОВКА ЗАДАЧИ	6
1.1 Предметная область	6
1.2 Задачи практики	6
2 РЕЗУЛЬТАТЫ РАБОТЫ.....	7
2.1. Изучение принципов работы генетических алгоритмов.....	7
2.2. Изучение особенностей задачи решения sudoku	7
2.3. Разработка плана по решению задачи.....	8
2.4. Выбор инструментов для реализации задачи.....	8
2.5. Разработка прототипа графического интерфейса программы	9
2.6. Реализация генетического алгоритма	11
2.7. Реализация графического интерфейса программы.....	14
2.8. Тестирование программы	15
3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ	17
4 ОТЗЫВ РУКОВОДИТЕЛЯ	18
ЗАКЛЮЧЕНИЕ	19
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	20

ВВЕДЕНИЕ

Генетические алгоритмы применяются для поиска решений в оптимизационных задачах с большим числом возможных вариантов. Целью данной работы является разработка программы, демонстрирующей решение sudoku при помощи генетического алгоритма. В ходе работы были изучены основные принципы работы генетических алгоритмов, разработаны способы представления и оценки решений, реализованы компоненты для инициализации, отбора, скрещивания и мутации особей, а также создан графический интерфейс для настройки параметров алгоритма и наблюдения за процессом решения.

1 ПОСТАНОВКА ЗАДАЧИ

1.1 Предметная область

Предметной областью данной работы является разработка программы для решения задач при помощи генетических алгоритмов. В качестве практического примера рассматривалась задача решения sudoku с набором заранее размещенных цифр.

1.2 Задачи практики

Основной целью данной работы является разработка приложения, демонстрирующего работу генетического алгоритма на примере решения sudoku.

Для достижения данной цели были определены следующие задачи:

1. Изучение принципов работы генетических алгоритмов
2. Изучение особенностей задачи решения sudoku
3. Разработка плана по решению задачи
4. Выбор инструментов для реализации задачи
5. Разработка прототипа графического интерфейса программы
6. Реализация генетического алгоритма
7. Реализация графического интерфейса программы
8. Тестирование программы

2 РЕЗУЛЬТАТЫ РАБОТЫ

2.1. Изучение принципов работы генетических алгоритмов

На данном этапе были изучены основы работы генетических алгоритмов и их применение для решения задач оптимизации. В качестве основного источника использовалось учебно-методическое пособие Т. В. Панченко [1]. Рассмотрены механизмы представления хромосомы, скрещивания, мутации, методы отбора, жесткие и мягкие ограничения, а также преимущества и недостатки различных способов реализации данных механизмов.

В результате были определены основные этапы работы генетического алгоритма, которые необходимо реализовать в программе:

1. формирование начальной популяции
2. оценка качества решений
3. выбор родительских особей
4. скрещивание особей
5. мутация особей
6. исправление особей в соответствии с ограничениями задачи
7. повторение процесса до достижения критерия остановки

2.2. Изучение особенностей задачи решения sudoku

На данном этапе были изучены основные правила игры. Игровое поле является таблицей размера 9x9, содержащей цифры от 1 до 9. В таблице так же выделены квадратные блоки 3x3 содержащие 9 клеток каждый. В верном решении каждый столбец, каждая строка и каждый блок должны содержать все возможные цифры без повторений.

Для представления игрового поля был выбран двумерный массив, позволяющий получать и изменять значения в клетках за константное время.

Для оценки решений была выбрана функция качества, основанная на количестве нарушений правил sudoku. За одно нарушение считается повторяющаяся цифра в строке, столбце или блоке, при этом нарушение в

одной клетке считается максимум один раз. Если нарушений нет, решение считается корректным. Формула функции качества: $\exp(-0.1 * \text{количество нарушений})$

2.3. Разработка плана по решению задачи

На данном этапе был составлен план разработки приложения. Была определена последовательность реализации программы и её основные компоненты.

Основными модулями программы являются:

1. Модуль работы с полем sudoku
2. Модуль генетического алгоритма
3. Модуль отрисовки графического интерфейса

Разработанный план выполнения задачи включает следующие этапы:

1. Разработка модели игрового поля sudoku
2. Реализация правил валидации и подсчета ошибок для sudoku
3. Разработка структуры генетического алгоритма
4. Реализация элементов генетического алгоритма для генерации, селекции и мутации хромосом
5. Реализация функции качества для задачи sudoku
6. Создание прототипа графического интерфейса программы
7. Интеграция графического интерфейса и генетического алгоритма
8. Добавление визуализации процесса решения
9. Добавление конфигурационных параметров в графический интерфейс
10. Тестирование программы на различных исходных данных

2.4. Выбор инструментов для реализации задачи

На данном этапе были определены основные инструменты, которые будут использоваться для реализации поставленной задачи.

Для создания приложения был выбран язык программирования Java. Данный язык подходит для решения задачи, так как он поддерживает объектно-ориентированное программирование и позволяет разделить программу на классы и модули, а также имеет средства для реализации графического интерфейса и тестирования.

Для разработки графического интерфейса была выбрана библиотека JavaFX, предоставляющая набор компонентов для разработки оконных приложений. Для проектирования и редактирования структуры графического интерфейса применялась программа Scene Builder.

В качестве среды разработки использовалась IntelliJ IDEA. Данная среда предоставляет инструменты для написания, сборки, отладки и тестирования java кода. Для сборки проекта и управления зависимостями был выбран Maven.

Для сокращения количества шаблонного кода применялась библиотека Lombok, предоставляющая набор аннотаций для автоматической генерации стандартных методов класса, таких как Getter, Setter, Builder, Constructor и других служебных элементов.

Для хранения данных использовалась библиотека Jackson Databind, позволяющая записывать и читать объекты в формате JSON.

Для хранения и контроля версий проекта использовалась система Git. Для организации удаленного репозитория был использована платформа GitHub.

2.5. Разработка прототипа графического интерфейса программы

На данном этапе был разработан прототип графического интерфейса программы. Были определены основные элементы, которые должны присутствовать в графическом интерфейсе:

1. Поле sudoku, содержащее информацию о фиксированных значениях в клетках, и значениях, установленных выбранной хромосомой генетического алгоритма.

2. Отображение графика изменения функции качества от номера популяции.
3. Область для настройки параметров поля sudoku, генетического алгоритма и визуализации.
4. Область, содержащая информацию о текущем состоянии генетического алгоритма.
5. Отдельное окно для ручного ввода поля sudoku.

В результате был создан прототип графического интерфейса, представленный на рис. 1 и рис. 2:

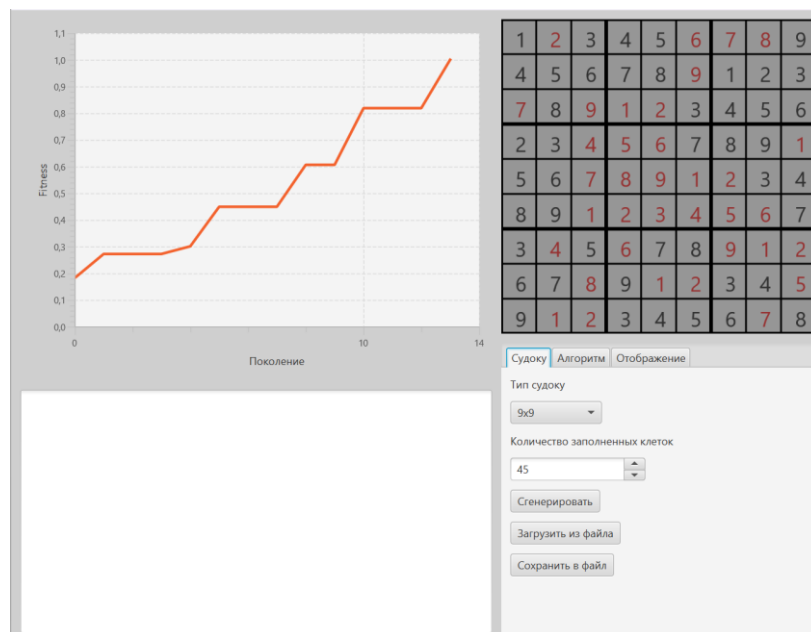


Рисунок 1 – Прототип графического интерфейса. Основное окно.

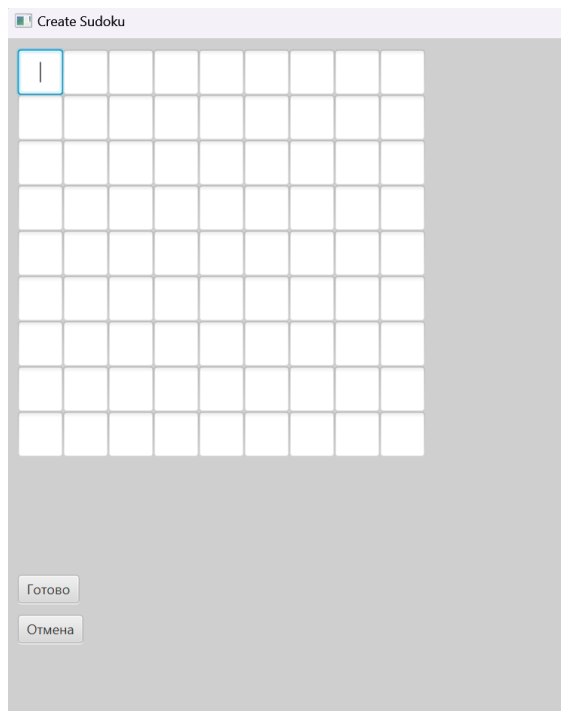


Рисунок 2 – Прототип графического интерфейса. Окно ручного ввода sudoku.

2.6. Реализация генетического алгоритма

В основе реализованного генетического алгоритма лежит класс `GeneticAlgorithm<DNA>`, не зависящий от конкретного способа представления решения. Основные компоненты вынесены в отдельные интерфейсы:

- `GAIinitializer` – формирует начальную популяцию.
- `GASelector` – осуществляет отбор особей для следующего поколения.
- `GACrossover` – отвечает за скрещивание особей.
- `GAMutate` – осуществляет мутацию особей.
- `GATask` – содержит методы вычисления функции качества, проверки условия остановки и коррекции генотипа.

Параметры работы алгоритма хранятся в классе `GAConfig`. Перед запуском алгоритма осуществляется проверка конфигурации на корректность. Реализованы следующие конфигурационные параметры:

- `generations` – максимальное число поколений до остановки алгоритма.
- `populationSize` – число особей в одном поколении.

- `copyBestRate` – доля особей с наибольшим значением функции качества, копируемых в следующее поколение.
- `mutationRate` – вероятность мутации особи.
- `crossoverRate` – вероятность скрещивания особей.
- `stagnationGenerations` – число поколений при застое до повторной инициализации особей
- `stagnationKeepRate` – доля особей с наибольшим значением функции качества, сохраняемых при стагнации.
- `similarityPunishment` – коэффициент штрафа за сходство.
- `similarityCompare` – число особей с наибольшим значением функции качества, проверяемых на сходство.
- `similaritySkip` – число особей с наибольшим значением функции качества без штрафа.
- `similarityThreshold` – минимальный коэффициент сходства, при котором накладывается штраф.
- `delayBetweenGenerationsMs` – задержка между расчетом поколений в миллисекундах.
- `randomSeed` – зерно генератора случайных чисел, используемое для осуществления детерминированных запусков.

На каждой итерации выполняются оценка и сортировка популяции, сохранение особей с наибольшим значением функции качества, отбор родителей, скрещивание и мутация потомков. Оценка особей производится в многопоточном режиме, используя все доступные процессорные ядра.

В качестве генотипа одной особи был выбран массив целых чисел от 1 до 9, характеризующий решение sudoku. Каждое число сопоставляется значению клетки поля sudoku. Генотип был разделен на блоки, представляющие собой столбцы поля sudoku, каждый из которых не содержит повторяющихся элементов.

Инициализация начальной популяции выполнялась с учетом выбранного представления генотипа. Для каждого экземпляра популяции формировался список целых чисел, разбитый на блоки по столбцам. Далее каждый блок заполнялся числами от 1 до 9, после чего значения внутри блока случайным образом перемешивались.

Была реализована мутация обменом. При выполнении мутации случайным образом выбирается блок, а затем внутри него выбираются две позиции. Если выбранные позиции различаются, значения в них меняются местами. Количество выполняемых перестановок зависит от заданной пользователем частоты мутации. Чем выше значение параметра, тем больше мутаций может быть произведено над генотипом.

Для отбора особей для следующего поколения был использован ранжированный метод отбора. Перед выполнением отбора особи упорядочиваются по заранее вычисленному значению функции качества. После этого каждой особи назначается вес в зависимости от ее позиции в полученном списке. Особи с более высоким рангом получают больший вес и имеют более высокую вероятность быть выбранными для дальнейшего скрещивания.

Скрещивание было реализовано при помощи равномерного обмена блоками между двумя родительскими особями. Для первого потомка с равным шансом выбирался блок из первого или второго родителя, для второго выбирался блок генов того родителя, который не попал в первого потомка.

Для того, чтобы алгоритм не сходил к единственному экстремуму, который может не являться верным решением, был реализован параллельный механизм образования ниш. На решения, находящиеся близко к друг другу накладывается штраф. Штраф вычисляется следующим образом:

1. Определенное количество особей с наибольшим значением функции качества не получают штраф, чтобы избежать возможного исключения наиболее качественных решений.

2. Для всех остальных особей производится сравнение с заданным количеством представителей поколения, имеющих наибольшие значения функции качества, за исключением самой оцениваемой особи.
3. Вычисляется коэффициент сходства особей путем подсчета относительного количества совпадающих чисел.
4. Из полученных значений берется максимальное, и накладывается штраф по формуле $\text{score} / (1 + \text{similarity} * \text{punishment})$, где score – исходный результат особи, similarity – вычисленный коэффициент сходства, punishment – заданный коэффициент штрафа.

2.7. Реализация графического интерфейса программы

Для реализации графического интерфейса была выбрана библиотека JavaFX. Данная библиотека предоставляет набор компонентов для создания оконных приложений. Для редактирования графического интерфейса применялась программа Scene Builder.

Расположение и стиль элементов графического интерфейса задаются при помощи FXML-файлов, что позволило отделить описание внешнего вида приложения от программной логики. Для реализации кнопок и конфигурационных полей были использованы классы JavaFX Button и Spinner соответственно. Отображение вкладок осуществляется при помощи элемента TabPane. Для отрисовки графика зависимости используется LineChart. Текстовое поле, отображающее информацию о текущей генерации реализовано при помощи TextArea. Отображение поля sudoku производится при помощи вызова методов класса GraphicsContext, связанного с Canvas. Для контроля положения элементов окна используются GridPane и VBox.

Для ввода поля sudoku используется GridPane, с элементами TextField для каждой клетки, динамически создаваемыми при открытии окна. Содержимое ячеек считывается, проверяется на корректность и записывается в матрицу sudoku при нажатии на кнопку Готово.

Обработка ввода пользователя и динамическое изменение состояния интерфейса были реализованы в специальных классах-контроллерах MainGuiController.java и CreateSudokuGuiController.java. Реализованы основное окно приложения и окно ручного ввода sudoku. В результате был создан интерфейс, изображенный на рис. 3 и рис. 4.

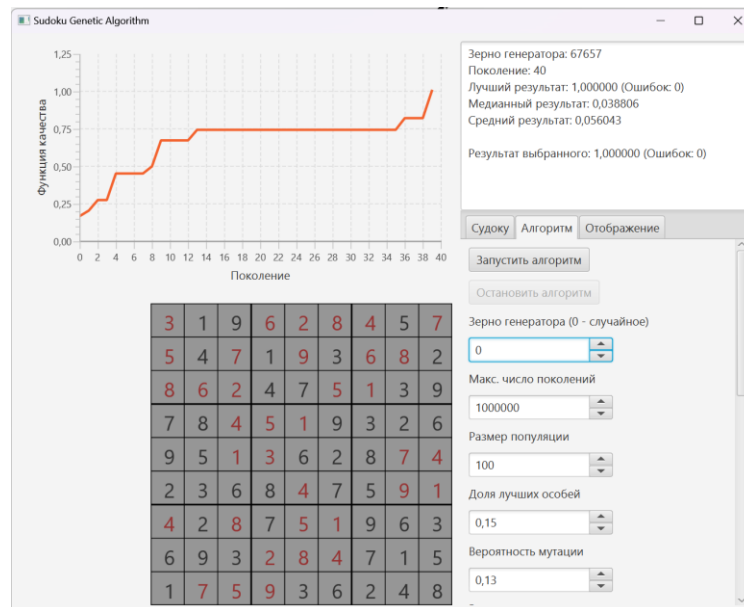


Рисунок 3 – Графический интерфейс. Основное окно.

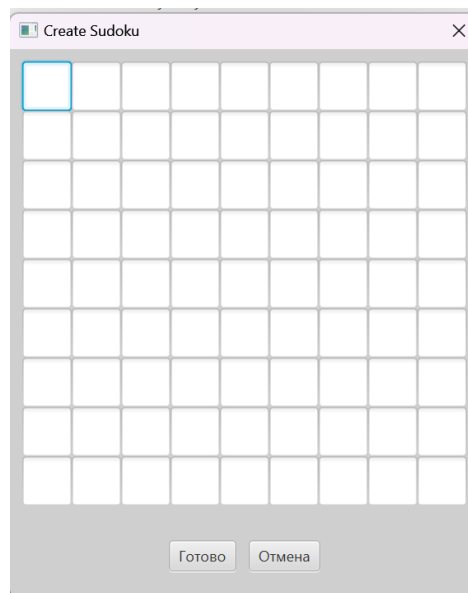


Рисунок 4 – Графический интерфейс. Окно ручного ввода sudoku.

2.8. Тестирование программы

Было произведено тестирование программы на различных входных данных. Проверялась корректность загрузки и ручного ввода поля sudoku, обработка пустых и некорректных значений, запуск алгоритма, обновление графика и отображение текущего состояния вычислений. Работа алгоритма проверялась на sudoku различной сложности. Для каждого запуска контролировались сохранение исходных значений, соответствие найденного решения правилам sudoku и корректность критерия остановки. Дополнительно исследовалось влияние размера популяции, вероятности мутации, числа поколений и параметров штрафа за сходство на скорость поиска решения. Повторные запуски с одинаковым зерном генератора случайных чисел давали одинаковый результат, что подтвердило воспроизводимость вычислений. В результате тестирования критических ошибок обнаружено не было.

3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

В ходе выполнения учебной практики для разработки программного обеспечения использовался язык программирования Java. В качестве среды сборки и управления зависимостями использовалась система Apache Maven.

Были использованы следующие библиотеки:

1. Lombok

Библиотека для автоматической генерации шаблонного кода, позволяющая сократить объем исходного кода и повысить его читаемость.

2. Jackson Databind

Библиотека для работы с данными в формате JSON. Применяется для чтения и записи на диск данных программы в читаемом формате.

3. JavaFX

Библиотека, предоставляющая набор компонентов для создания оконных приложений и различных элементов интерфейса.

В качестве интегрированной среды разработки использовалась IntelliJ IDEA. Данная среда предоставляет инструменты для написания, сборки, отладки и тестирования программного кода.

Для отслеживания изменений исходного кода применялась система контроля версий Git, позволяющая сохранять историю изменений файлов и предоставляющая возможность возврата к предыдущим версиям проекта при необходимости.

Для хранения исходного кода и организации удаленного репозитория использовалась платформа GitHub.

4 ОТЗЫВ РУКОВОДИТЕЛЯ

По результатам работы учебная практика оценена на .

ЗАКЛЮЧЕНИЕ

В результате выполнения учебной практики были изучены основные принципы работы генетических алгоритмов и особенности их применения для решения задач оптимизации, была разработана программа на языке программирования Java для решения sudoku при помощи генетического алгоритма, а также создан интерактивный графический интерфейс при помощи библиотеки JavaFX. Было проведено тестирование программы и графического интерфейса на различных входных данных. Исходный код программы размещён в репозитории GitHub [2].

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Панченко, Т. В. Генетические алгоритмы : учебно-методическое пособие / Т. В. Панченко ; под ред. Ю. Ю. Тарасевича. – Астрахань : Издательский дом «Астраханский университет», 2007. – 87, [3] с.
2. Баженов, К. LETI_Practice: репозиторий с исходным кодом разработанной программы [Электронный ресурс]. – URL: https://github.com/KonstantinBazhenov/LETI_Practice